Step 1: Design the Embedded Data Model

```
{
 _id: ObjectId(),
 name: "Alice Smith",
 email: "alice@example.com",
 orders: [
  {
   orderId: ObjectId(),
   orderDate: ISODate("2023-10-26T10:00:00Z"),
   totalAmount: 120.00,
   items: [
    {
     productId: ObjectId(),
     productName: "Laptop",
     quantity: 1,
     price: 120.00
    }
   ]
  }
 ]
}
```

Step 2: Insert Embedded Data

```
ecommerce> db.customers.insertOne({
|   name: "Bob Johnson",
|   email: "bob@example.com",
|   orders: [
|     {
|       orderId: new ObjectId(),
|       orderDate: new Date("2023-11-15T14:30:00Z"),
|       totalAmount: 75.50,
|       items: [
|         { productId: new ObjectId(), productName: "Mouse", quantity: 1, price: 25.00 },
|         { productId: new ObjectId(), productName: "Keyboard", quantity: 1, price: 50.50 }
|       ]
|     },
|     {
|       orderId: new ObjectId(),
|       orderDate: new Date("2023-11-20T09:00:00Z"),
|       totalAmount: 30.00,
|       items: [
|         { productId: new ObjectId(), productName: "USB Cable", quantity: 2, price: 15.00 }
|       ]
|     }
|   ]
| })
|
{
  acknowledged: true,
  insertedId: ObjectId('69d0b75176e2c68af13682d6')
}
```

```
ecommerce> db.customers.find({ name: "Bob Johnson" })
[
  {
    _id: ObjectId('69d0b75176e2c68af13682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69d0b75176e2c68af13682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69d0b75176e2c68af13682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69d0b75176e2c68af13682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
```

Step 3: Referenced Data Model (Design)

```
// Example 1
{
  _id: ObjectId(),
  name: "Charlie Brown",
  email: "charlie@example.com"
}
// Example 2
{
  _id: ObjectId(),
  customerId: ObjectId("..."),
  orderDate: ISODate("2023-11-22T12:00:00Z"),
  totalAmount: 90.00,
  items: [
    {
      productId: ObjectId(),
      productName: "Headphones",
      quantity: 1,
```

```
    price: 90.00
  }
 ]
}
```

Step 4: Insert Data Using the Referenced Model

```
ecommerce> db.customers.insertOne({
| name: "David Lee",
| email: "david@example.com"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69d0b89376e2c68af13682d7')
}
ecommerce> let customer = db.customers.findOne({ name: "David Lee" });
| let customerId = customer._id;
|

ecommerce> db.orders.insertOne({
|   customerId: customerId,
|   orderDate: new Date("2023-11-25T16:00:00Z"),
|   totalAmount: 55.00,
|   items: [
|     { productId: new ObjectId(), productName: "Webcam", quantity: 1, price: 55.00 }
|   ]
| })
|
{
  acknowledged: true,
  insertedId: ObjectId('69d0b8a276e2c68af13682d9')
}
ecommerce> db.orders.insertOne({
|   customerId: customerId,
|   orderDate: new Date("2023-11-28T11:00:00Z"),
|   totalAmount: 110.00,
|   items: [
|     { productId: new ObjectId(), productName: "Monitor", quantity: 1, price: 110.00 }
|   ]
| })
|
{
  acknowledged: true,
  insertedId: ObjectId('69d0b8ab76e2c68af13682db')
}
```

```
ecommerce> db.orders.find({ customerId: customerId })
[
  {
    _id: ObjectId('69d0b8a276e2c68af13682d9'),
    customerId: ObjectId('69d0b89376e2c68af13682d7'),
    orderDate: ISODate('2023-11-25T16:00:00.000Z'),
    totalAmount: 55,
    items: [
      {
        productId: ObjectId('69d0b8a276e2c68af13682d8'),
        productName: 'Webcam',
        quantity: 1,
        price: 55
      }
    ]
  },
  {
    _id: ObjectId('69d0b8ab76e2c68af13682db'),
    customerId: ObjectId('69d0b89376e2c68af13682d7'),
    orderDate: ISODate('2023-11-28T11:00:00.000Z'),
    totalAmount: 110,
    items: [
      {
        productId: ObjectId('69d0b8ab76e2c68af13682da'),
        productName: 'Monitor',
        quantity: 1,
        price: 110
      }
    ]
  }
]
```

Step 5: Query Embedded Model

```
ecommerce> db.customers.find({ name: "Bob Johnson" })
[
  {
    _id: ObjectId('69d0b75176e2c68af13682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69d0b75176e2c68af13682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69d0b75176e2c68af13682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69d0b75176e2c68af13682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
```

```
ecommerce> db.customers.aggregate([
|   {
|     $match: {
|       "orders.orderDate": { $gt: new Date("2023-11-16T00:00:00Z") }
|     }
|   }
| ])
|
[
  {
    _id: ObjectId('69d0b75176e2c68af13682d6'),
    name: 'Bob Johnson',
    email: 'bob@example.com',
    orders: [
      {
        orderId: ObjectId('69d0b75176e2c68af13682d1'),
        orderDate: ISODate('2023-11-15T14:30:00.000Z'),
        totalAmount: 75.5,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d2'),
            productName: 'Mouse',
            quantity: 1,
            price: 25
          },
          {
            productId: ObjectId('69d0b75176e2c68af13682d3'),
            productName: 'Keyboard',
            quantity: 1,
            price: 50.5
          }
        ]
      },
      {
        orderId: ObjectId('69d0b75176e2c68af13682d4'),
        orderDate: ISODate('2023-11-20T09:00:00.000Z'),
        totalAmount: 30,
        items: [
          {
            productId: ObjectId('69d0b75176e2c68af13682d5'),
            productName: 'USB Cable',
            quantity: 2,
            price: 15
          }
        ]
      }
    ]
  }
]
```

Step 6: Query Referenced Model

```
ecommerce> db.customers.find({ name: "David Lee" })
[
  {
    _id: ObjectId('69d0b89376e2c68af13682d7'),
    name: 'David Lee',
    email: 'david@example.com'
  }
]
```

```
ecommerce> db.orders.find({ customerId: customerId })
[
  {
    _id: ObjectId('69d0b8a276e2c68af13682d9'),
    customerId: ObjectId('69d0b89376e2c68af13682d7'),
    orderDate: ISODate('2023-11-25T16:00:00.000Z'),
    totalAmount: 55,
    items: [
      {
        productId: ObjectId('69d0b8a276e2c68af13682d8'),
        productName: 'Webcam',
        quantity: 1,
        price: 55
      }
    ]
  },
  {
    _id: ObjectId('69d0b8ab76e2c68af13682db'),
    customerId: ObjectId('69d0b89376e2c68af13682d7'),
    orderDate: ISODate('2023-11-28T11:00:00.000Z'),
    totalAmount: 110,
    items: [
      {
        productId: ObjectId('69d0b8ab76e2c68af13682da'),
        productName: 'Monitor',
        quantity: 1,
        price: 110
      }
    ]
  }
]
```

```
ecommerce> db.customers.aggregate([
|   { $match: { name: "David Lee" } },
|   {
|     $lookup: {
|       from: "orders",
|       localField: "_id",
|       foreignField: "customerId",
|       as: "orders"
|     }
|   }
| ])
|
[
  {
    _id: ObjectId('69d0b89376e2c68af13682d7'),
    name: 'David Lee',
    email: 'david@example.com',
    orders: [
      {
        _id: ObjectId('69d0b8a276e2c68af13682d9'),
        customerId: ObjectId('69d0b89376e2c68af13682d7'),
        orderDate: ISODate('2023-11-25T16:00:00.000Z'),
        totalAmount: 55,
        items: [
          {
            productId: ObjectId('69d0b8a276e2c68af13682d8'),
            productName: 'Webcam',
            quantity: 1,
            price: 55
          }
        ]
      },
      {
        _id: ObjectId('69d0b8ab76e2c68af13682db'),
        customerId: ObjectId('69d0b89376e2c68af13682d7'),
        orderDate: ISODate('2023-11-28T11:00:00.000Z'),
        totalAmount: 110,
        items: [
          {
            productId: ObjectId('69d0b8ab76e2c68af13682da'),
            productName: 'Monitor',
            quantity: 1,
            price: 110
          }
        ]
      }
    ]
  }
]
```

Step 7: Update Operations

```
ecommerce> db.customers.updateOne(
|   { name: "Bob Johnson", "orders.orderId": ObjectId("661abc1234def567890abcd1") },
|   { $set: { "orders.$.totalAmount": 80.00 } }
| )
|
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

```
ecommerce> db.orders.updateOne(
|   { _id: ObjectId("661abc1234def567890abcd1") },
|   { $set: { totalAmount: 60.00 } }
| )
|
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Step 8: Delete Operations

```
ecommerce> db.customers.updateOne(
|   { name: "Bob Johnson" },
|   { $pull: { orders: { orderId: ObjectId("661abc1234def567890abcd1") } } }
| )
|
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

```
ecommerce> db.orders.deleteOne({ _id: ObjectId("661abc1234def567890abcd1") })
{ acknowledged: true, deletedCount: 0 }
```

Observations and Analysis

In this lab, a comparison was conducted between embedded and referenced data models in MongoDB within the context of an e-commerce scenario. The embedded model stores related data within a single document, which makes it highly efficient for queries that require retrieving all related information at once. For example, fetching a customer along with all their associated orders can be done in a single query without the need for joins. This significantly improves read performance and reduces query complexity.

However, the embedded model also has several limitations. One major drawback is that the document size can grow very large as more related data is added, which may impact performance and exceed MongoDB document size limit. Additionally, updating specific parts of deeply nested data can become more complex and less efficient, especially when only a small portion of the document needs to be modified. Data duplication in embedded models can also lead to potential consistency issues if updates are not handled carefully.

On the other hand, the referenced model separates data into multiple collections, such as customers and orders. This approach provides greater flexibility and scalability, especially for applications with complex relationships or frequently changing data. Although retrieving related data requires additional operations such as $lookup, it allows for better data normalization and easier updates.

From a performance perspective, the embedded model is generally faster for simple read operations, while the referenced model offers better maintainability and scalability. Therefore, the choice between these models should be based on application requirements, including data size, update frequency, and query complexity.

Selection Justification

For a large scale Ecommerce application, I would recommend using the referenced data model. This is because large systems typically involve complex data relationships, high volumes of transactions, and frequent updates. By separating data into different collections, the referenced model allows for better data organization and easier maintenance.

In addition, this approach minimizes data duplication, which helps maintain consistency across the database. Updates can be performed more efficiently since only specific documents need to be modified rather than an entire embedded structure. Although querying data may require more complex operations, such as using the $lookup operator, this can be optimized through proper indexing and query design.

Overall, the referenced model provides greater scalability, flexibility, and long-term efficiency, making it more suitable for modern, large-scale applications.